Tailwind v4

Type:

React + Vite | TypeScript 5.9 | Local-First AI | Tauri Roadmap | Privacy-First | No Backend (Alpha)

PROJECT STATE REPORT

ZaraOS

AI-Native Desktop Operating Environment Prototype

Version:

- ST_AI.md, TAURI_ROADMAP.md

Layer	Technology	Notes
Monorepo	pnpm workspaces	Node.js 24, 3 workspace artifacts
Language	TypeScript 5.9	Strict mode, zero errors
Frontend	React 19 + Vite 7	Tailwind CSS v4, shadcn/ui
API Server (future)	Express 5 + pino	Built and running, not yet wired to frontend
Database (future)	PostgreSQL + Drizzle ORM	Schema defined, not yet used
Build	esbuild (API), Vite (frontend)	Static SPA output
Deployment	Replit static hosting	SPA rewrite /* -> /index.html

The largest structural change to date. The Alpha 0.1 mocked AIEngine class was retired and replaced with a full, layered AI Runtime: 28 new TypeScript files across 6 sub-layers inside src/core/ai/.

28	6	6	0
NEW AI FILES	PROVIDERS	BUILT-IN TOOLS	TS ERRORS

03 /

Architecture -- Full Layer Diagram

```
USER INPUT (voice / gesture / keyboard / plugin)
        |
        v
  useRuntime()   <--  All UI components call this exclusively
        |
        v
  ZaraRuntime    <--  zara-runtime.ts (OS brain, 452 lines)
    |   |
    |   +-- permission check  -->  deny-by-default gate
    |
    +-- parseAndRoute()  -->  CommandRouter  -->  ParsedCommand
    +-- Skills Layer     -->  skillRuntime.executeSkill()
    +-- System Layer     -->  mocked (Tauri invoke() in future)
    +-- Plugin Layer     -->  registerPlugin() / executeCommand from plugin
    +-- AI Layer         -->  aiRuntime.sendMessage() | streamMessage()
              |
              v
        AI Runtime (ai-runtime.ts, 319 lines)
              |
              +-- buildSystemPrompt()     <-- zara-system-prompt.ts
              +-- buildContextBlock()     <-- context-injector.ts
              |     +-- system-context.ts    (CPU, RAM, time, OS state)
              |     +-- privacy-context.ts   (current permission snapshot)
              |     +-- skills-context.ts    (active skills list)
              |
              +-- conversationMemory      <-- conversation-memory.ts
              |     +-- memory-storage.ts    (localStorage persistence)
              |
              +-- requestRouter.dispatch  <-- request-router.ts
              |     +-- provider-router.ts   (select by permissions + config)
              |     +-- model-router.ts      (select model by capabilities)
              |
              +-- provider dispatch       <-- provider-adapter.ts (interface)
                    +-- local-provider.ts     (default, no network)
                    +-- ollama-provider.ts    (ready, one flag to enable)
                    +-- llamacpp-provider.ts  (ready, disabled)
                    +-- openai-provider.ts    (cloud-gated)
                    +-- anthropic-provider.ts (cloud-gated)
                    +-- gemini-provider.ts    (cloud-gated)
```

Full Source File Inventory

05 / Core -- OS Brain

File	Lines	Purpose
core/types.ts	--	All shared TypeScript interfaces (ParsedCommand, ZaraStatus, PluginManifest, PermissionCategory, InputSource, AIProvider...)
core/zara-runtime.ts	452	Central OS brain. Routes all input through permission checks, dispatches to AI / system / skill / plugin layers
core/runtime-context.tsx	128	React context + useRuntime() hook. Bridges runtime to UI
core/permissions.ts	--	Deny-by-default manager. Categories: mic, camera, cloud_ai, network, files, system_actions
core/input-mode.tsx	--	Voice / Keyboard / Gesture / Hybrid mode state, persisted to localStorage

Core -- AI Runtime Layer (28 files, Alpha 0.3)

File	Purpose
core/ai/ai-runtime.ts	Central AI orchestrator (319 lines). Single entry point for all AI ops from ZaraRuntime
core/ai/models/ai-capabilities.ts	Capability enum: streaming, tool-calling, vision, long-context, fast-inference
core/ai/providers/provider-adapter.ts	IProviderAdapter interface + AIStreamChunk / AIStreamCallback streaming types
core/ai/providers/local-provider.ts	Default simulated provider. Realistic latency, intent-aware responses, token streaming simulation
core/ai/providers/ollama-provider.ts	Full Ollama REST API (localhost:11434). Health check, streaming, model listing. Ready -- one flag to enable
core/ai/providers/llamacpp-provider.ts	llama.cpp server REST API integration. Ready -- disabled by default
core/ai/providers/openai-provider.ts	OpenAI API. Cloud-gated -- requires cloud_ai permission + user API key
core/ai/providers/anthropic-provider.ts	Anthropic Claude API. Cloud-gated
core/ai/providers/gemini-provider.ts	Google Gemini API. Cloud-gated
core/ai/memory/memory-types.ts	MemoryEntry, MemoryStats, ConversationMessage interfaces. Security rules documented inline
core/ai/memory/memory-storage.ts	localStorage persistence layer for conversation history
core/ai/memory/conversation-memory.ts	Context window management, token estimation (1 token ~= 4 chars), history pruning
core/ai/routing/provider-router.ts	Selects provider based on permission state + user config
core/ai/routing/model-router.ts	Selects model given provider + required capabilities
core/ai/routing/request-router.ts	Dispatches to provider adapter, handles retries, propagates streaming callbacks
core/ai/context/context-injector.ts	Assembles full context block injected into every prompt (system + privacy + skills)

core/ai/context/system-context.ts	CPU, RAM, time, privacy mode snapshot surfaced to AI
core/ai/context/privacy-context.ts	Current permission states injected into prompt so Zara knows what it can/cannot do
core/ai/context/skills-context.ts	Active skills list surfaced to AI for capability awareness
core/ai/tools/tool-types.ts	ZaraTool interface with parameters, permissions, dangerous flag, version
core/ai/tools/tool-registry.ts	Registry class + 6 built-in tool declarations
core/ai/tools/tool-executor.ts	Tool call dispatch layer
core/ai/prompts/zara-system-prompt.ts	Full Zara personality, capability map, security rules, OS context, tone guidelines

Core -- Skills Layer

File	Purpose
core/skills/types.ts	SkillManifest, SkillResult interfaces
core/skills/skill-runtime.ts	Skill executor -- routes executeSkill() calls to built-in implementations
core/skills/builtin-skills.ts	Built-in skill declarations: timer, web search, file browser, alarms, reminders, weather

Library Layer -- Engine Integration Points

File	Purpose
lib/command-router.ts	parseAndRoute() -- NL intent parser returning structured ParsedCommand with intent, confidence, requiresPermission, destructive flags
lib/voice-engine.ts	Web Speech API / Whisper.cpp / Vosk integration point. Wire real audio capture in Alpha 0.4
lib/gesture-engine.ts	MediaPipe Hands integration point. Wire RAF loop + camera stream in Alpha 0.4
lib/gesture-mapper.ts	Gesture-to-command mapping
lib/privacy-store.tsx	Privacy state React context (mic, camera, cloud AI, network, local AI status)
lib/ai-engine.ts	Alpha 0.1 legacy engine. No longer imported -- orphaned, retained for reference only

Pages (11 panels)

Page	What it does
home.tsx	Dashboard -- live clock, CPU/RAM/network stats, Privacy Fortress widget, System Activity feed
assistant.tsx (482 lines)	Zara AI chat -- real token streaming UI (character-by-character), ZaraStatus indicators, voice button, provider info bar, memory token counter, clear button
console.tsx	Natural language command console -- structured intent routing, command history, ParsedCommand display

apps.tsx	App launcher grid with voice command hints
files.tsx	File browser placeholder with mocked file tree
media.tsx	Audio/video player placeholder
settings.tsx	System configuration panel
privacy.tsx	Permission toggle panel -- mic, camera, cloud AI, network, local AI with live state
ai-providers.tsx (316 lines)	Provider manager -- 6 providers, API key input (localStorage only), AI Runtime Status widget, memory stats, clear memory button
developers.tsx	Developer portal -- plugin registry, full PluginManifest spec, 4 example plugins, Zara Store preview
skills.tsx	Skills browser -- view, enable/disable built-in skills

Components

Component	Purpose
layout.tsx	OS shell -- sidebar with 11 panel links, input mode indicator, command box trigger, system status
error-boundary.tsx	React class ErrorBoundary. Double-wrapped: outer covers providers, inner covers Router. Shows Retry + Restart UI instead of blank screen
ai-runtime-status.tsx	Live widget: active provider, model ID, latency (ms), memory token count, simulated/real badge
global-command-box.tsx	Ctrl+Space command palette wired to ZaraRuntime
input-mode-indicator.tsx	Sidebar widget showing Voice / Keyboard / Gesture / Hybrid mode live
confirmation-dialog.tsx	Modal gate for all destructive runtime actions
ui/ (40+ components)	Full shadcn/ui component library

Security Model (Alpha 0.3)

Six tools are declared in the tool registry. They are architecture-ready -- execution stubs exist and will be wired to real implementations in Alpha 0.4.

Permission Categories

As of Alpha 0.3

Category	Default	Controls
mic	OFF	Voice engine microphone access
camera	OFF	Gesture engine camera access
cloud_ai	OFF	All cloud provider API calls
network	OFF	Web search, external fetch
files	OFF	File read/write tool calls
system_actions	OFF	System command tool calls

```
            -> onChunk(delta) called per token -> UI appends char-by-char
            -> conversationMemory.add()  <- persist turn to localStorage
            -> broadcast AIRuntimeStatus to all listeners
    -> ZaraStatus: idle -> thinking -> streaming -> idle
    -> ai-runtime-status.tsx widget updates live
```

Command Console Flow

```
User types command in console.tsx
  -> runtime.executeCommand(text)
  -> parseAndRoute(text) -> ParsedCommand { intent, confidence, requiresPermission, destructive }
  -> permission check -> deny if permission missing
  -> if destructive -> ConfirmationDialog shown
  -> dispatch to system / skill / AI layer
  -> return CommandResult { success, output, message }
```

Provider Selection Flow

```
requestRouter.dispatch(request)
  -> providerRouter.selectProvider()
      -> check: is cloud_ai permission granted?
      -> check: does user have an API key configured?
      -> check: is provider healthy? (health check endpoint)
      -> fall back to local-provider if any check fails
  -> modelRouter.selectModel(providerId, requiredCapabilities)
  -> providerAdapter.send() | providerAdapter.stream()
```

08 /

Documentation Library (19 files in artifacts/zaraos/docs/)

File	Contents
ARCHITECTURE.md	Layer diagram, command flow, design decisions
SECURITY.md	Permission system, threat model, key storage
ROADMAP.md	Alpha -> Beta -> v1.0 feature plan
PLUGIN_SPEC.md	Full PluginManifest spec -- id, name, version, developer, permissions, voiceCommands, gestureCommands, aiCapabilities, sandboxRequired
LOCAL_FIRST_AI.md	Local-first AI philosophy and provider stack
AI_RUNTIME_ARCHITECTURE.md	Alpha 0.3 AI Runtime layer diagram and data flows
AI_SECURITY_MODEL.md	AI-specific security: prompt injection, key storage, cloud gating
LOCAL_AI_ROADMAP.md	Ollama, Whisper.cpp, llama.cpp, MediaPipe integration plan
MEMORY_MODEL.md	Conversation memory design, token estimation, storage format
TOOL_CALLING_ARCHITECTURE.md	Tool registry, executor, permission gating for tool calls
ZARA_PERSONALITY.md	Zara's personality, tone guidelines, response rules
TAURI_ROADMAP.md	Web app -> native desktop app -> Linux ISO full plan
SECURITY_AUDIT_ALPHA_0_1.md	Completed security audit for Alpha 0.1
SKILLS_ARCHITECTURE.md	Skills system design and built-in skill declarations
COMMAND_CONFIRMATION_MODEL.md	Destructive action confirmation flow
GITHUB_EXPORT_GUIDE.md	Instructions for exporting to GitHub
LINUX_ISO_PREP.md	Linux ISO preparation notes
REPLIT_INDEPENDENCE_AUDIT.md	Audit of Replit-specific dependencies and removal path
CONNECTED_SERVICES_ROADMAP.md	Future backend services integration plan

Boilerplate and Gotchas

- [illegible] is used exclusively for the React web app layer.

IMPORTANT CONSTRAINT

Beta — Desktop App

Alpha 0.4 -- Real Engine Integration

11 /

12 /